

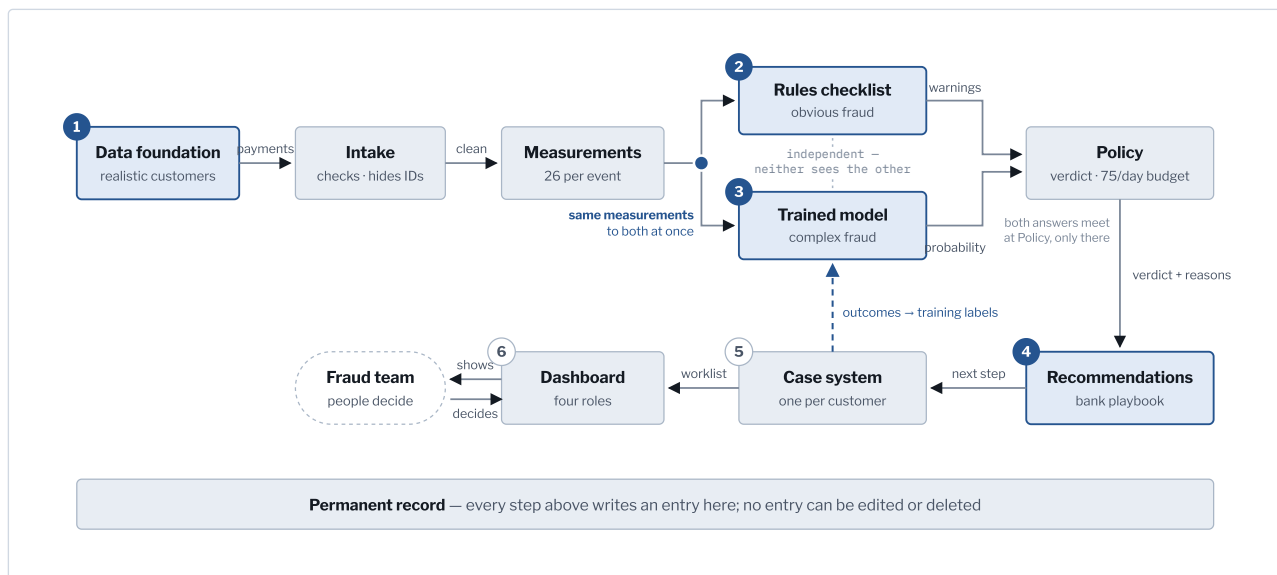
Risk Radar Blueprint

What we are building, in six parts. What each part is for, what it must do, and the number that tells us whether it is doing it.

Scope statement. Risk Radar watches a Nigerian bank's stream of payments, spots the ones that look like fraud, and puts them in front of a fraud analyst with a clear recommendation of what to do next. It **advises**; a person decides. It never blocks, holds or reverses a payment.

It is built from six parts: **realistic data** to learn from, a **checklist of written rules** for fraud that is obvious, a **trained model** for fraud that is not, a **recommendation engine** that turns the evidence into a next step, a **case system** with a permanent record, and a **dashboard** the fraud department works in all day.

How the parts connect



Every payment runs left to right along the top, and the result runs back along the bottom to the people who act on it. At the fork (the blue dot), the *same* measurements go to the rules and to the model at once. The rules read all twenty-six; the model reads twenty-one, because the five about money coming in made it worse at fraud it had not seen, so they are kept for the rules (D78b). Four measurements were added on 17 September for the Nigerian fraud types below: a region new to the customer, card uses in the hour, other customers paying the same destination, and the local hour (D82). Since 11 September the intake also takes sign-in, device, SIM and payee events and incoming credits (D72, D78), which is where those extra measurements come from. Neither waits for the other or sees the other's answer, which is what lets us measure what each one catches on its own. Their two answers meet only at the policy step, which decides; they are never blended into one number. The dashed blue line is the loop that matters most in the long run. When an analyst records "real fraud" or "false alarm", that answer becomes a training label, which is how the model eventually stops learning only from invented data.

Data foundation — a realistic financial dataset, built with context

Objective: give the rules and the model believable Nigerian payment behaviour to learn from, with fraud that hides among ordinary life rather than standing apart from it.

No bank's real transactions are available, and no public Nigerian fraud dataset exists. So we generate customers — ordinary people, salaried workers, shop owners, traders — who each own accounts, keep habits and pay people, then have some of them robbed in three different ways. Real published statistics shape the result: our fraud rate is tuned to the one Nigeria's interbank settlement body reports.

Must do

- Model customers owning several accounts, so one person's incident stays one investigation.
- Give legitimate customers the behaviours fraud also has — paying new people, paying many people in an hour, occasional large sums — so fraud has something to hide in.
- Fail its own check if any single measurement can separate fraud from normal by itself.

MEASURE	TARGET	NOW	STATUS
Fraud rate	≈ 0.30% (NIBSS)	0.279%	MET
Fraud incidents to test on	≥ 600	1,455 · seven kinds	MET
Measurements that give fraud away on their own	0 over 15%	0 on the first twelve; three new ones separate their fraud type by definition	STATED (D82C)
Checked against published channel and state figures	all key shares	fraud rate only	PARTLY

Rules checklist — fraud that is obvious

Objective: catch the fraud a person would recognise instantly, with near-certain accuracy, and catch it even when it is a new variant the model has never seen.

Plain written rules, each producing a named warning with its evidence — never a score. Some raise the risk (a burst of payments, a run of refused card attempts). Some override everything (a sanctioned destination). Some lower it (a long-standing payee receiving a normal amount). Settings change from the admin screen, with every change recorded. Three rules were added on 14–15 September, each tuned on the data before going live: a takeover sequence (two warning events such as a new device and a PIN change, then a payment to a new destination, D77b),

money fanning in to an account from many senders (D78), and the payment that moves that money on, the second leg (D79). Four more were added on 17 September for fraud Nigerian banks report losing money to (D82): a new account several customers paid today (the collection account of a scam), a new destination hours after the SIM changed, a long-dormant account moving a large sum, and a card used at terminal after terminal somewhere new. Each was tuned by a stated rule: the most of its own fraud type at no more than five false alerts a day.

Must do

- Explain itself: every warning says what fired and why.
- Stay inside the team's capacity: no rule may spend more of the daily alert budget than it earns.
- Be tested one rule at a time, and drop any rule the data does not support.

MEASURE	TARGET	NOW	STATUS
Card-testing rule: how often it is right	≥ 0.95	0.998	MET
Card testing caught, new variant, rules alone	≥ 0.90	165 of 165 · 1.00	MET
Payment-burst rule: share of daily budget	≤ 60%	~28% at 120 a day (5+ in an hour, to a destination new to the bank)	MET
New mule fan-out caught by the rules alone	as high as budget allows	0.94 (was 0.41)	MET
"Familiar payee" rule firing where there is no payee	never	never (was 84% of its firings)	FIXED
Mule-ring fraud value flagged, with the second leg	as high as budget allows	72.3% · seven types now share the budget (was 84.9% for three)	MET
SIM swap caught by the rules alone	as high as budget allows	0.85 · by construction partly (D82c)	MET
Scams caught by the rules alone	as high as budget allows	0.15 · group collections look the same	LEFT TO THE MODEL

MEASURE	TARGET	NOW	STATUS
Every rule measured on its own	all thirteen	ten	PARTLY

3

PART THREE

Trained model — fraud that is complex, calibrated to tell the truth

Objective: catch fraud that no single rule describes — the combination of a new device, a quiet pause and a rush of transfers — and give a probability that means exactly what it says.

Hundreds of small decision trees, each correcting the last, trained on older data and then adjusted on newer data it never saw, so that "0.30" really means about three in ten. It is one voice, not the decision: on its own it catches only what resembles its training, which is why the rules sit beside it.

Must do

- Be tested on a kind of fraud hidden from its training, not only on more of the same.
- Produce an honest probability, because the daily workload is predicted from it.
- Use the same measurements live as in training, checked by a test.

MEASURE	TARGET	NOW	STATUS
Account takeover caught, never seen in training	≥ 0.70	1.00 (rules alone 0.88, model alone 1.00)	MET · PARTLY WRITTEN IN (D77
Honesty of its probability (stated ÷ actual)	≥ 0.90	0.919 (15 Sep corpus)	MET
Calibration error where alerts are raised	≤ 0.04	0.032	MET
Same numbers in training and live	identical	identical, tested	MET
Useful on its own	—	0.63 across seven types, below the rules and model together (0.82)	BY DESIGN
Works on data somebody else made, retrained there	better than random	PaySim: 101.6× random; as shipped 1.9×	MUST BE RETRAINED PER BANK

Recommendation engine — the next step, in the bank's own procedure

Objective: turn the evidence into one clear next action and its reason — "block the card and reissue: 14 refused small attempts" — so an analyst never has to work out what to do from raw data.

It reads the verdict, the warnings, the money at risk and the customer's pattern, and picks the step from a playbook. It is deterministic and written down, never learned, so every recommendation can be explained in the words that produced it.

On "real bank policy". No bank has shared its fraud procedures with us, so the playbook today is our own, written from how Nigerian fraud desks describe their work. It is built as a lookup the bank can replace: their procedure in, their steps out, with no code change. We will not describe it as a bank's policy until one is loaded.

Must do

- Give one action, numbered next steps, and the reason, for every case.
- Say how urgent it is, tied to a response clock.
- Record whether the analyst followed it, so the advice can be checked against real outcomes.

MEASURE	TARGET	NOW	STATUS
Cases with an action, steps and reason	100%	100%	MET
Bank's own procedure loaded	yes	our playbook, bank-replaceable	PARTLY
How often analysts follow the advice	tracked weekly	recorded, not yet reported	NOT YET

Case system and permanent record — what holds it together

Objective: turn many alerts about one customer into one case, keep the team inside its capacity, and prove afterwards who did what.

- Group alerts about the same customer into one case, so an attacker moving between someone's accounts stays one investigation.
- Hold the whole system to the daily alert budget — rules first, the model gets what is left.
- Record every decision and action permanently; administrators may tune the system but can never open a case.
- Run the Nigerian clocks in working days, hold the 24-hour customer flag, and keep machine-handled cases apart from the analysts' queue (D71, D73, D80).

MEASURE	TARGET	NOW	STATUS
Time from payment to decision (95% of the time)	under 2 s	0.88 s (11 Sep)	MET · RE-RUN AFTER EVENTS
Payments lost in a 5× burst	0	0	MET

MEASURE	TARGET	NOW	STATUS
Alerts per day, whole system	≤ 75	72 in the rebuilt demo day (15 Sep, before D82)	MET
Analyst reviews a day, after machine actions	below 75	57.9 · detection unchanged (D82b)	SHADOW FIRST
False alerts per fraud incident	below 12–14	2.0 : 1	MET
Record edits possible	none	none · tamper-evident	MET

What the over-budget day turned out to be. An earlier demo day ran 28% over the budget. The cause was the demo builder, not the system: its live day was generated as a different bank with no history, and was counted on top of a day already loaded. With that fixed, the rebuilt demo day raised 82 alerts and caught both real incidents in it. That demo bank remembered only about three days, so the payment-burst rule fired more there than it would in a real bank; it now remembers 30 days (D76c). Rebuilt on 15 September, the live day raised 72 alerts against a budget of 75 and caught all three fraud incidents in it. Its thresholds come from the second half of the month, because the rules on incoming money need that long to learn what a normal day looks like (D78f).

6

PART SIX

Dashboard — where the department works

Objective: let each person in the fraud department do their day's job from one screen: analysts clear cases, the lead sees whether the desk is coping, security handles break-ins, the administrator tunes settings.

Fraud analyst

- The live desk: what is arriving, waiting and expected this hour
- The next case handed to them, largest risk first, with who else shares its destinations and devices
- The recommendation as a checklist of steps, each recorded with its result; escalation with a reason

Fraud operations lead

- The case tracker: every case by stage and how long it has waited
- How many alerts turned out to be false
- Closes cases whose outcome is recorded

Information security

- Their own escalated queue, with the analyst's reason
- Hands a case back with findings

Administrator

- Rules, thresholds, models, the record
- No access to any case, by design

MEASURE	TARGET	NOW	STATUS
Keystrokes to record an outcome	1	1	MET
A case followed from alert to closed, through escalation and back	every stage visible	tested as analyst, InfoSec and lead (D83)	MET
Cases past their response clock	under 10%	not yet measured with real users	NOT YET

MEASURE	TARGET	NOW	STATUS
Median time to decide a case	under 3 min	not yet measured	NOT YET

The number that sums it up

Shown a kind of fraud hidden from its training, with the team held to 75 alerts a day, the whole system catches **about eight incidents in ten** (0.82) across seven kinds of fraud: account takeover 1.00, mule fan-out 0.96, card testing 1.00, SIM swap 0.97, dormant accounts 0.87, scams 0.52, card cloning 0.43. The first three did not fall when the other four were added. It raises two false alerts for every fraud incident, where the best desks report twelve to fourteen, and flags 82% of the money stolen. Scams and cloned cards are the hard cases, because group payments and travellers look like them. Several of these figures are partly written in: our own program writes the warning signs our rules read (D77d, D82c).

On data we did not write, the picture is plainer. Retrained on PaySim, the model ranks fraud a hundred times better than chance; the model trained on our simulator, used as it ships, barely better than chance, so a bank must retrain it on its own outcomes. Two Nigerian transaction files turned out to have fraud labels no detector could learn from (D81a). A dataset with real chargeback labels, IEEE-CIS, is ready to run.

We can claim the method is sound and measured. We cannot yet claim it works on real criminals, because we have never seen any. Running it alongside a real bank for a month, alerting nobody, is what would settle that.

Out of scope

- Blocking, holding or reversing payments — advisory only, permanently. A directive or machine action is a recommendation the bank's own systems may choose to act on (D74, D80c).
- Connecting to a real bank's systems — a separate project.
- Detecting whole networks of linked criminals — a later version. The two-leg link, money in and the payment that moves it on, is built (D79).
- Testing a new model on live traffic before switching to it — designed, not built.

Who owns which part

PART	OWNER
1 · Data foundation, and testing each rule on the data	Ire
2, 3 · Rules and model code · 5 · intake, measurements, policy, case system, record	Chidera
4 · Recommendation playbook	Chidera, with the team
6 · Dashboard	Paul
Plan, tests, documents, presentation	Kanyinsola

audit.json and the load test. Each decision behind them is in the project decision log. Companions: *What We Are Building* and *What We Learned From Testing Both Approaches*.